

#6. A New Paradigm: TabPFN

Instructor: **Hankook Lee** @ Efficient Learning Lab., Sungkyunkwan University

Toward Tabular Foundation Models

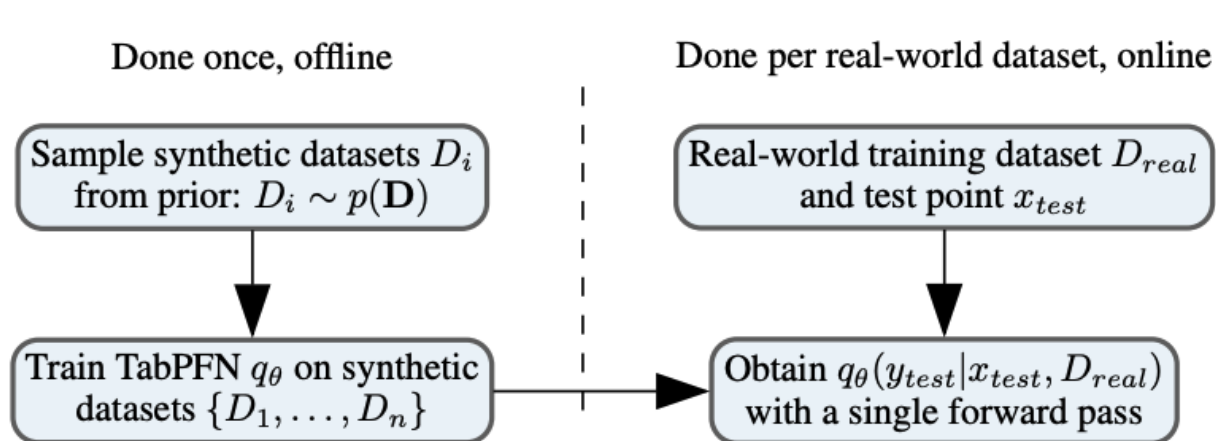
- Can we build a **tabular foundation model** that can transfer across many tables and tasks?
- Why is this difficult?
 - Tables are highly heterogeneous
 - Unlike text, tabular tasks are not naturally standardized
 - Large-scale training requires unifying many tables into a common format
- **Two directions:**
 - LLM-based tabular foundation models
 - **TabPFN-style foundation models** ← this lecture

Prior-Fitted Networks (PFNs)

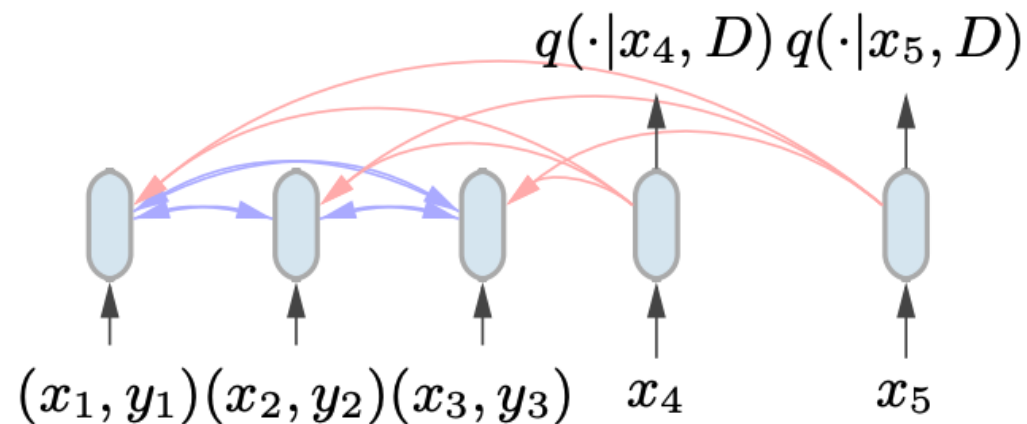
- **Core Idea:** Learn to solve new tasks by training on many synthetic tasks sampled from a predefined prior
 - $p(D)$: A predefined prior over diverse **data-generating distributions**
 - $D \sim p(D)$ is a sampled data-generating distribution ($\approx$ task)
 - Generate a training dataset $D_{\text{train}} = \{(x_i, y_i)\}_{i=1}^n$ and a test sample $(x_{\text{test}}, y_{\text{test}})$ from D
 - Learn a model $q_{\theta}(\cdot | x_{\text{test}}, D_{\text{train}})$ to predict y_{test} on diverse tasks
 - q_{θ} receives $(x_1, y_1), (x_2, y_2), \dots, (x_n, y_n)$ and x_{test} as input, and produces y_{test} as output
 - This form is similar to **in-context learning** & we can use **Transformer** for modeling q_{θ}
- Given any real-world dataset D_{real} and a test point x_{test} , the model q_{θ} can predict the test label y_{test} **without adaptation**

Prior-Fitted Networks (PFNs)

- **Core Idea:** Learn to solve new tasks by training on many synthetic tasks sampled from a predefined prior



(a) Prior-fitting and inference

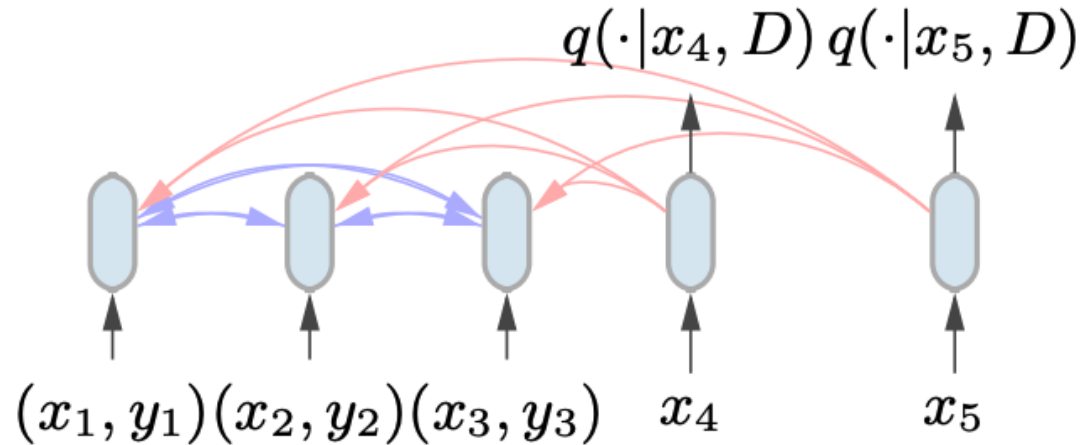


(b) Architecture and attention mechanism

- Given any real-world dataset D_{real} and a test point x_{test} , the model q_θ can predict the test label y_{test} **without adaptation**

TabPFN v1: Applying PFNs to Tables

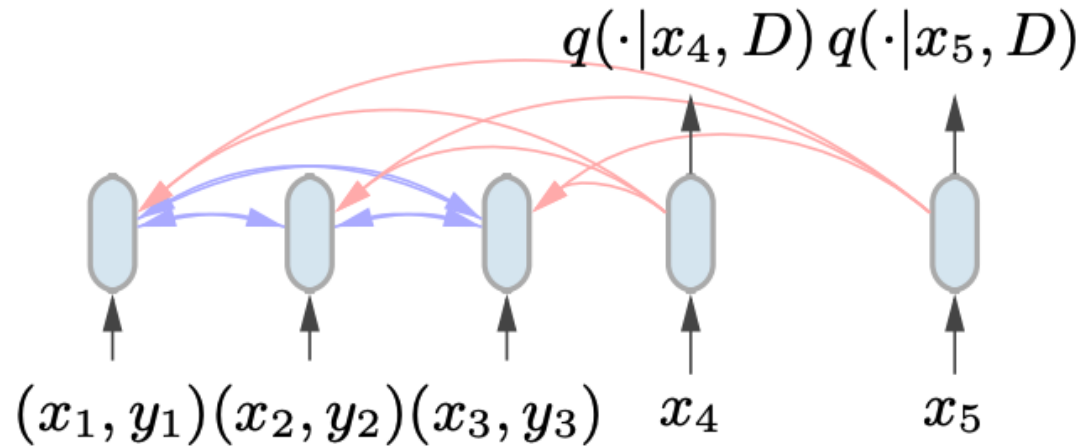
- **Key Idea:** A PFN for tabular prediction
 - **Each row is treated as one token:**
 - A training datapoint: $(x, y) \mapsto \text{MLP}_x(x) + \text{MLP}_y(y)$
 - A test input: $x \mapsto \text{MLP}_x(x) + \text{MLP}_y([\text{mask}])$
 - Zero-padding handles different numbers of columns



(b) Architecture and attention mechanism

TabPFN v1: Applying PFNs to Tables

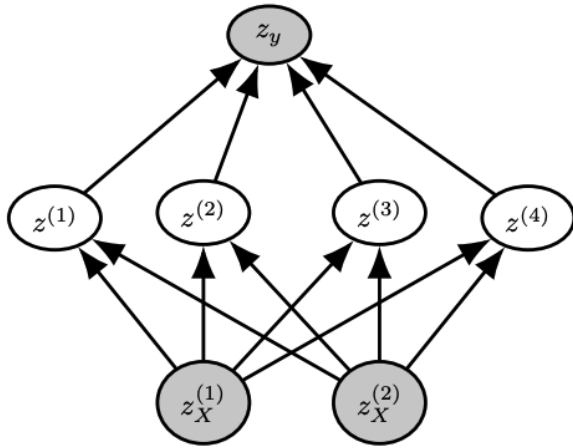
- **Key Idea:** A PFN for tabular prediction
 - **TabPFN Transformer architecture:**
 - Transformer processes all train/test row tokens together
 - Self-attention allows test samples to attend to training samples
 - Final hidden state of each test token is passed to an MLP classifier



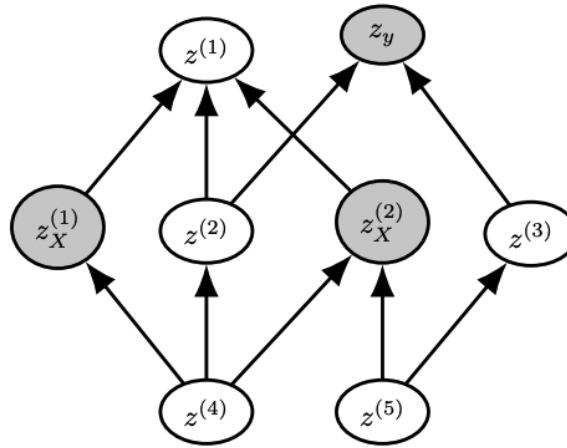
(b) Architecture and attention mechanism

TabPFN v1: Applying PFNs to Tables

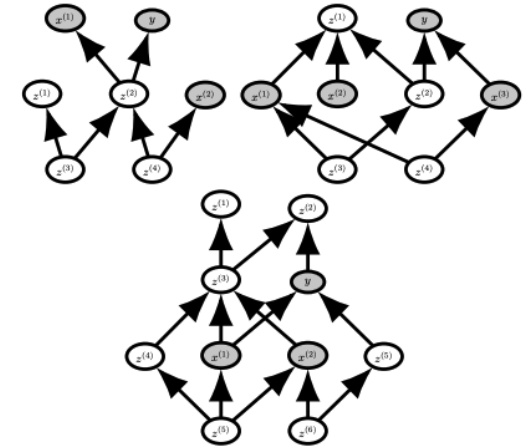
- **Key Idea:** A PFN for tabular prediction
 - **Synthetic priors for tabular data:**
 - **SCM:** select observed feature nodes and a target node from a random graph
 - **BNN:** sample a neural network and generate $\tilde{y}$ from x
 - Convert continuous targets into class labels



(a) A BNN



(b) An SCM



(c) SCMs sampled from the prior

TabPFN v1: Applying PFNs to Tables

- **Key Idea:** A PFN for tabular prediction
 - **Transformer architecture**
 - **Synthetic priors for tabular data**
- **Why It works?**
 - The synthetic prior is rich enough to cover most real datasets approximately
 - Transformers are flexible enough function approximators to learn the Bayesian posterior for that prior
 - Full row-attention lets the model adapt instantly to new training data

TabPFN v1: Applying PFNs to Tables

- **TabFPN** outperforms MLP and GBDT baselines on small datasets and achieves the fastest runtime

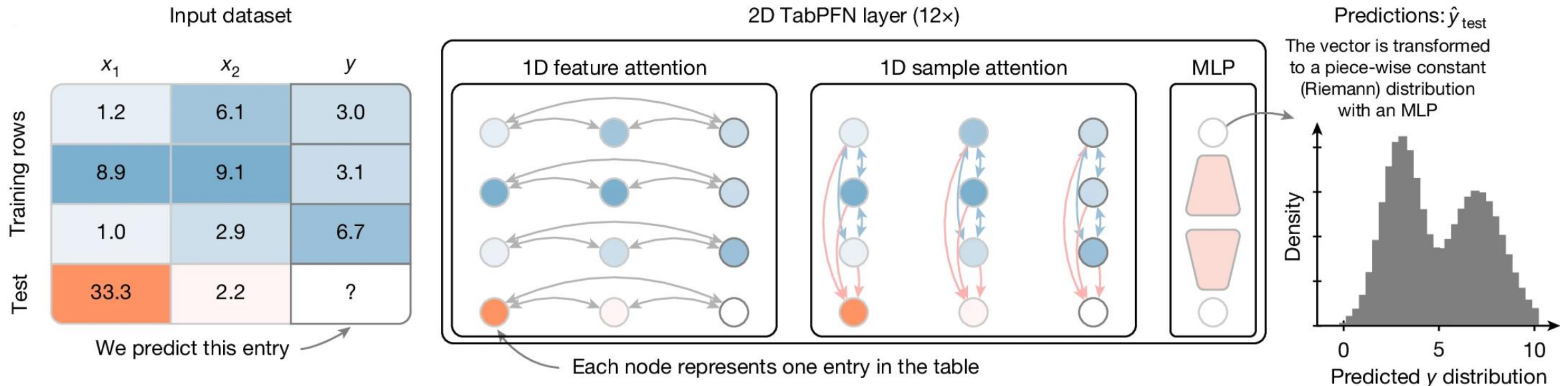
	LightGBM	CatBoost	XGBoost	ASKL2.0	AutoGluon	TabPFN _{n.e.}	TabPFN	TabPFN + AutoGluon
M. rank AUC OVO	6.9722	4.9444	6.1944	4.4722	4	3.8056	2.9444	2.6667
Mean rank Acc.	6.8889	4.9722	6.0556	5.1667	3.8889	3.8889	2.8889	2.25
Mean rank CE	5.7778	5.4444	6	6.4167	3.1111	4.1389	3.0278	2.0833
Mean AUC OVO	0.92±.013	0.924±.011	0.924±.01	0.929±.0096	0.93±.0091	0.932±.0088	0.934±.0086	0.934±.0084
Mean Acc.	0.862±.012	0.864±.011	0.866±.011	0.87±.014	0.881±.01	0.873±.0095	0.879±.0089	0.886±.0094
Mean CE	0.75±.039	0.747±.029	0.759±.04	0.813±.073	0.714±.014	0.727±.021	0.716±.019	0.711±.014
Mean time (s) (Tune + Train + Predict)	3280	3746	3364	3601	3077	1.301 (CPU) 0.0519 (GPU)	37.59 (CPU) 0.6172 (GPU)	3109 (CPU) 3077 (GPU)

TabPFN v1: Limitations

- **Do not consider 2D structure of tabular data**
 - TabPFN v1 treats each row as a single token
- **Model constraints**
 - Less than 1K training samples and 100 columns
 - Multiclass classification with less than 10 classes
- **Require manual preprocessing** for missing and categorical values

TabPFN v2

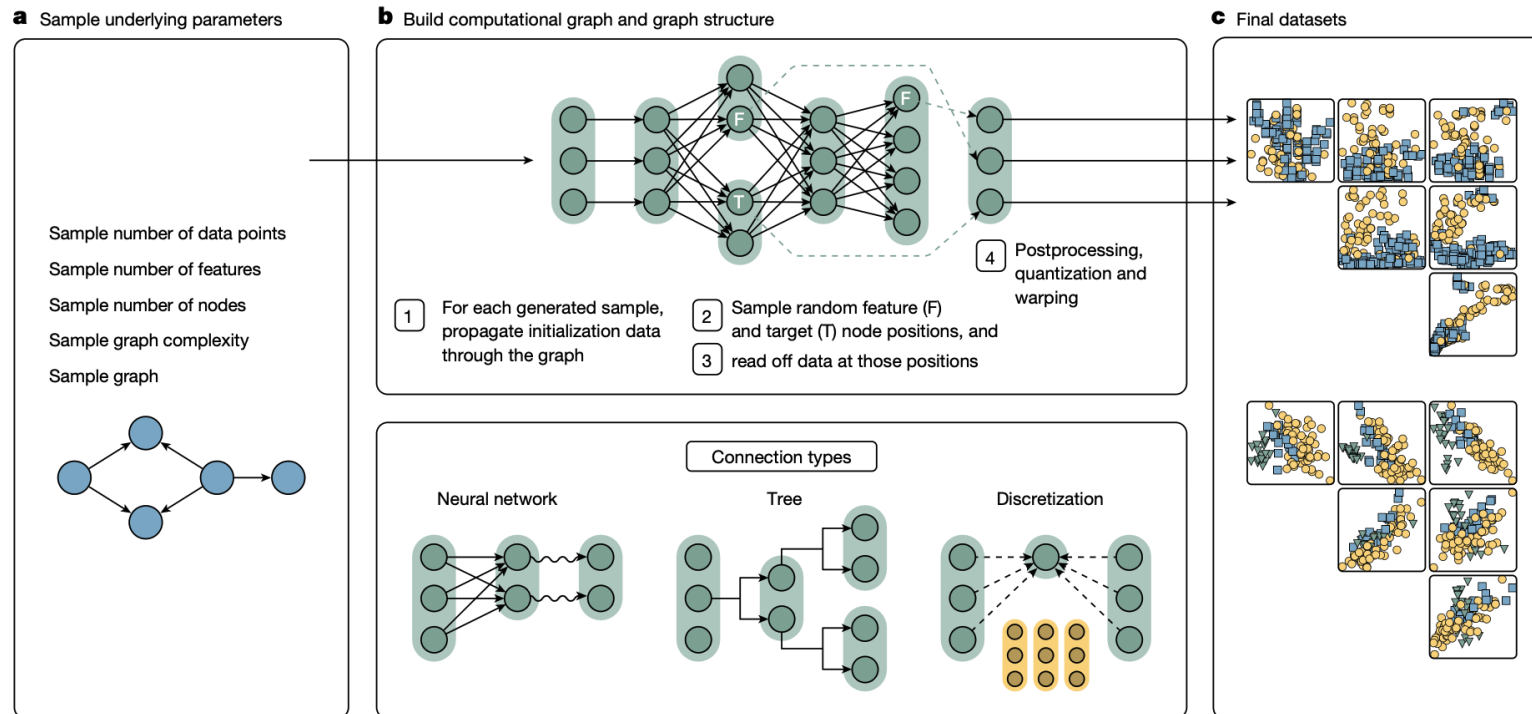
- **Key change #1: From Row Tokens to 2D Table Modeling**
 - **Represent each cell as one token** to better handle 2D tabular structure and feature heterogeneity
 - **Apply 2D attention:** feature-wise & sample-wise attention alternatively



TabPFN v2

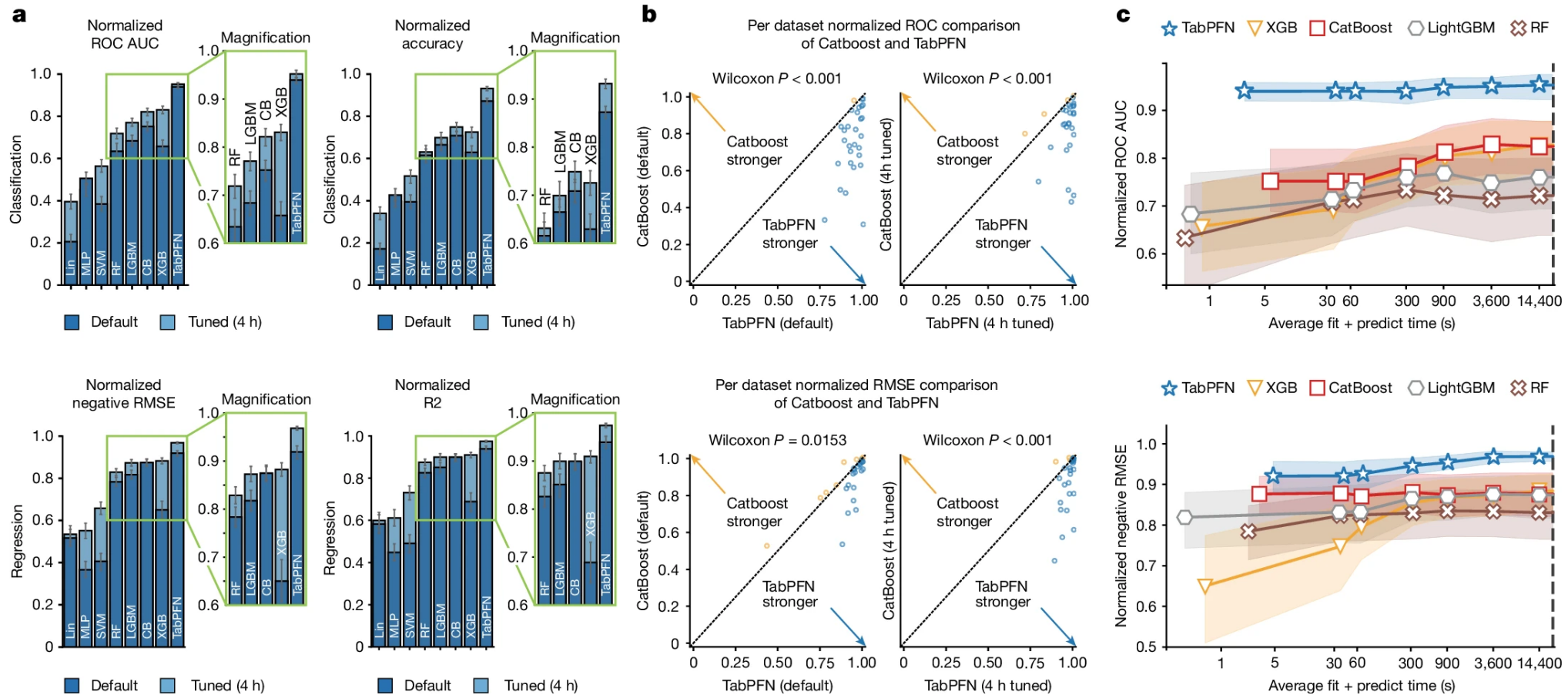
- **Key change #2: Richer Synthetic Prior**

- TabPFN v2 is trained on synthetic tasks that better reflect real-world tables
 - Synthetic task components: categorical features, missing values, quantization, ...
 - Datasets up to 10,000 samples and 500 features



TabPFN v2

- **TabPFN v2** outperforms MLP and GBDT baselines on small datasets



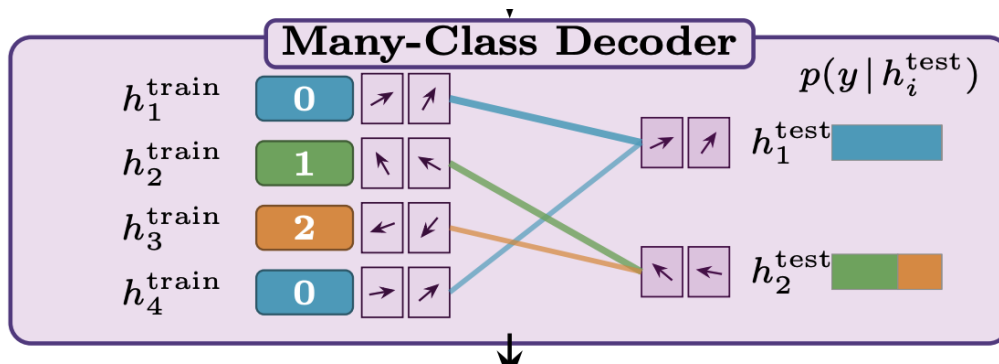
TabPFN v2: Limitations

- **Computational cost**
 - Cell-level representations and two-way attention increase inference cost
- **Memory scalability**
 - Processing larger context tables requires more memory
- **Task scale**
 - Classification evaluation is limited to tasks with up to 10 classes

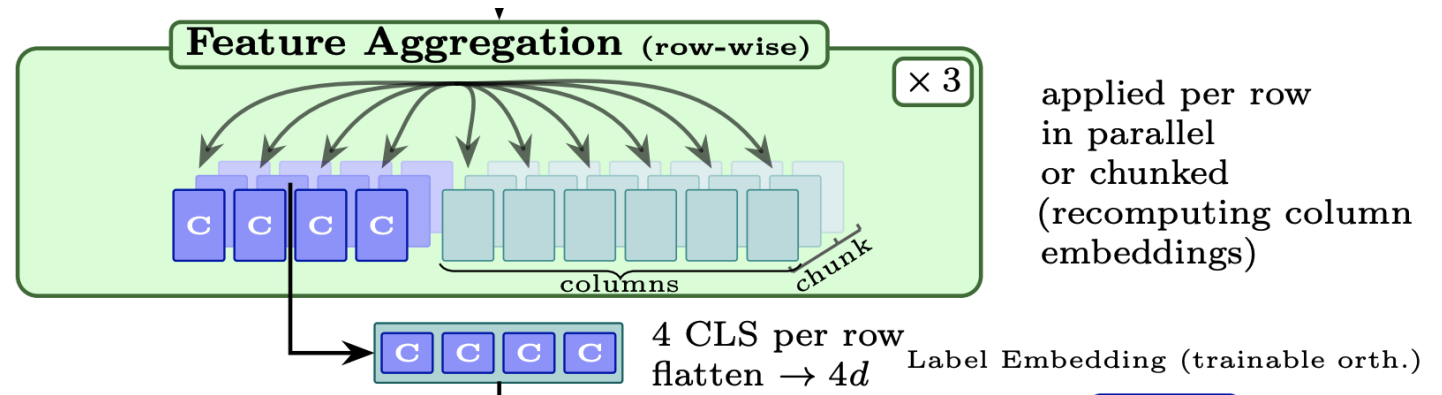
TabPFN v3

- **Key changes:**

- **Row compression:** Aggregate feature-level information into compact row embeddings before in-context learning
- **Retrieval-based prediction:** Replace the fixed classification head with an attention-based soft nearest-neighbor decoder

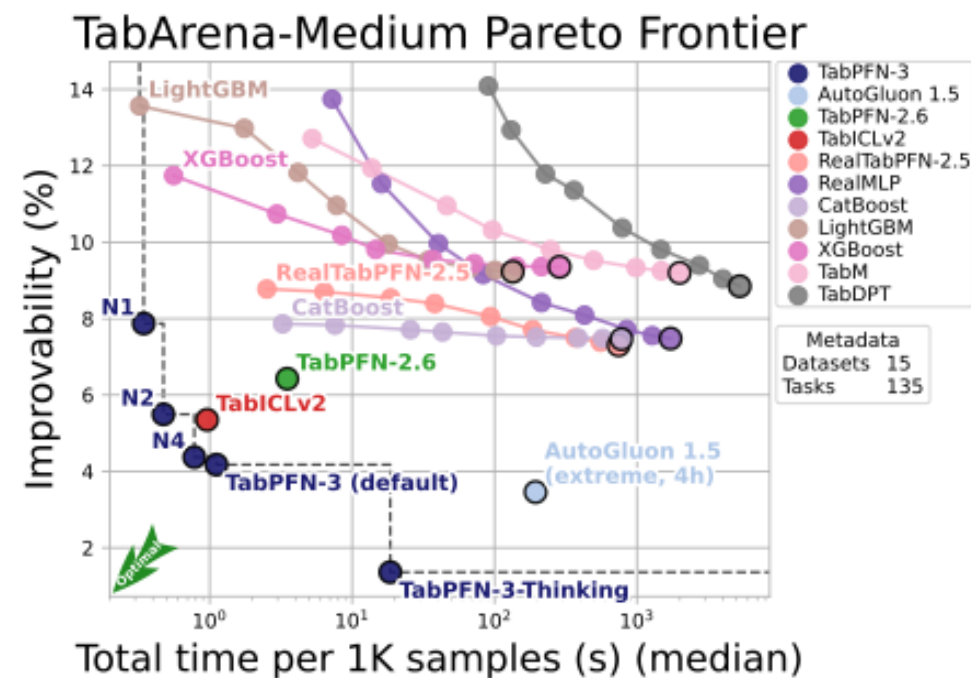
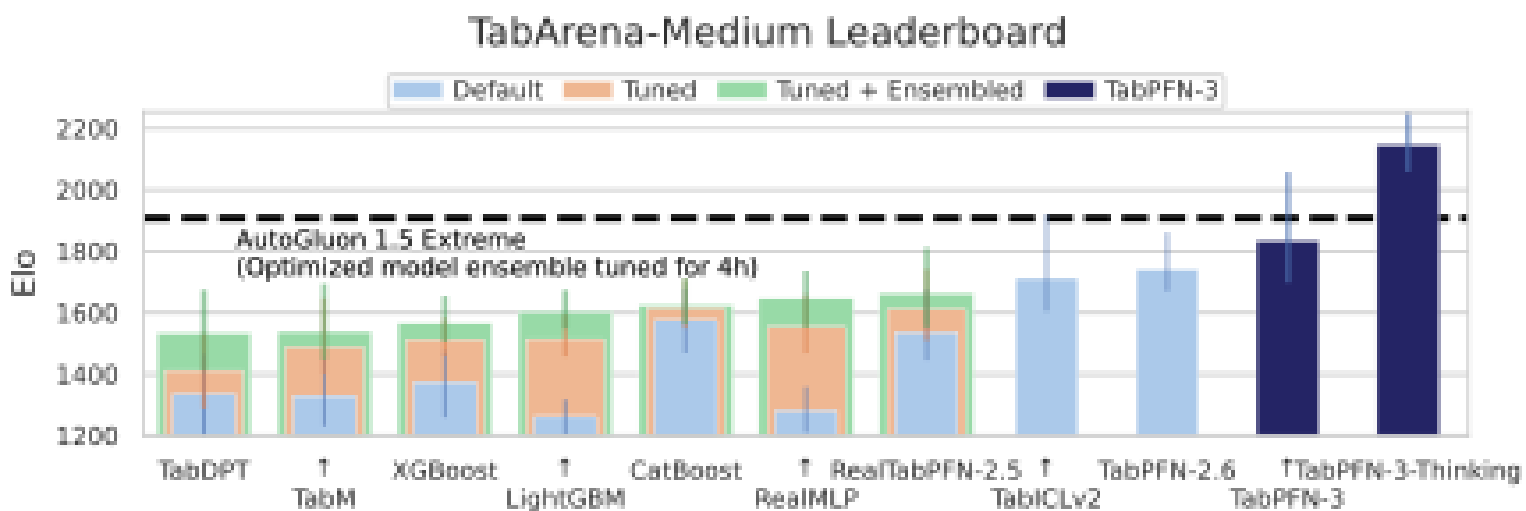


Attention-weighted
average of one-hot
encoded labels (y_i)



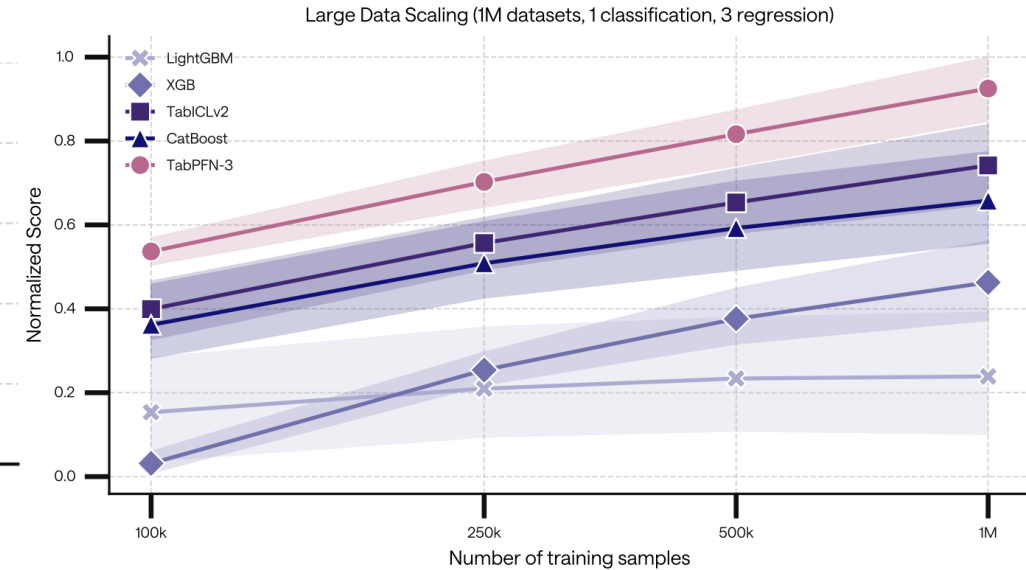
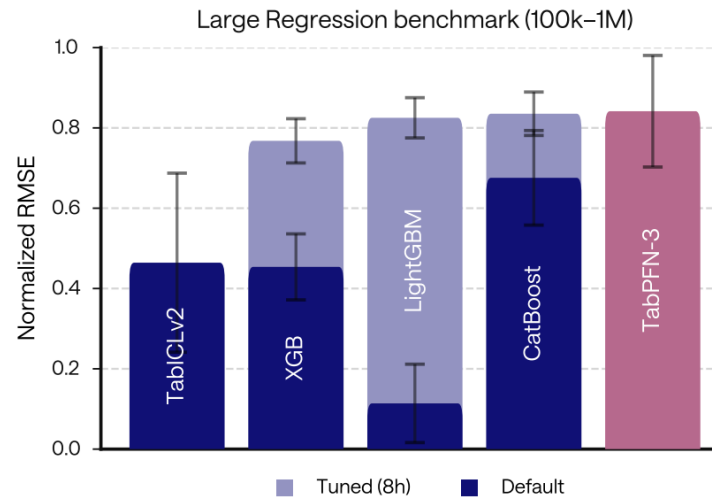
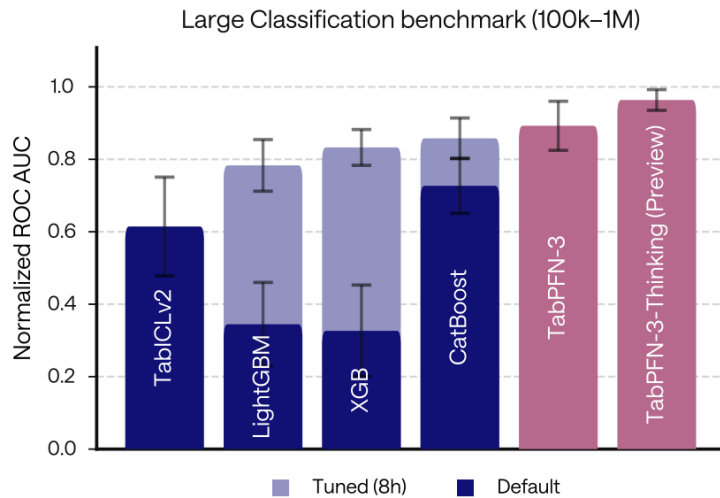
TabPFN v3

- **TabPFN v3** achieves SOTA performance with fast inference speed



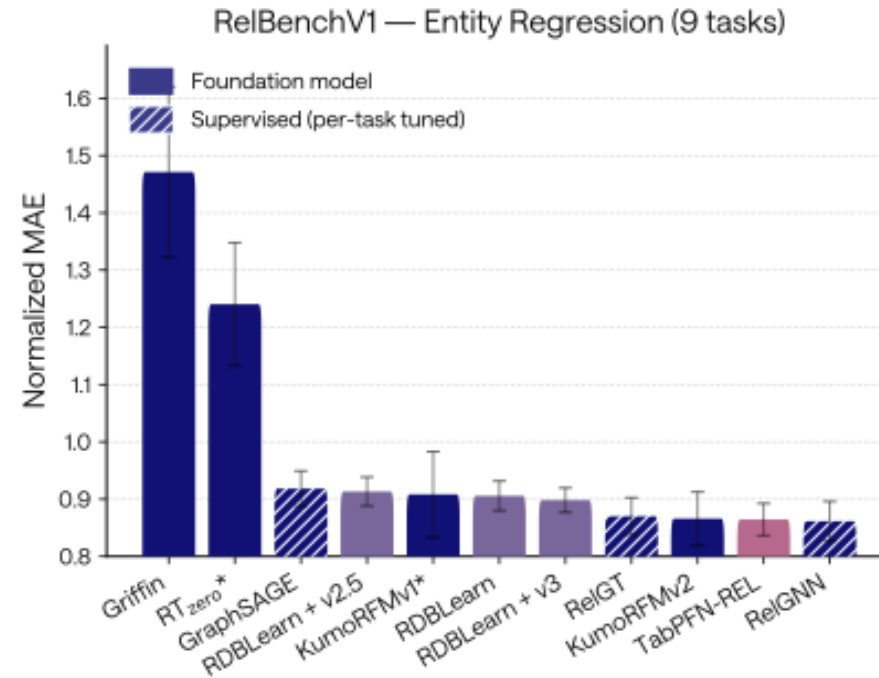
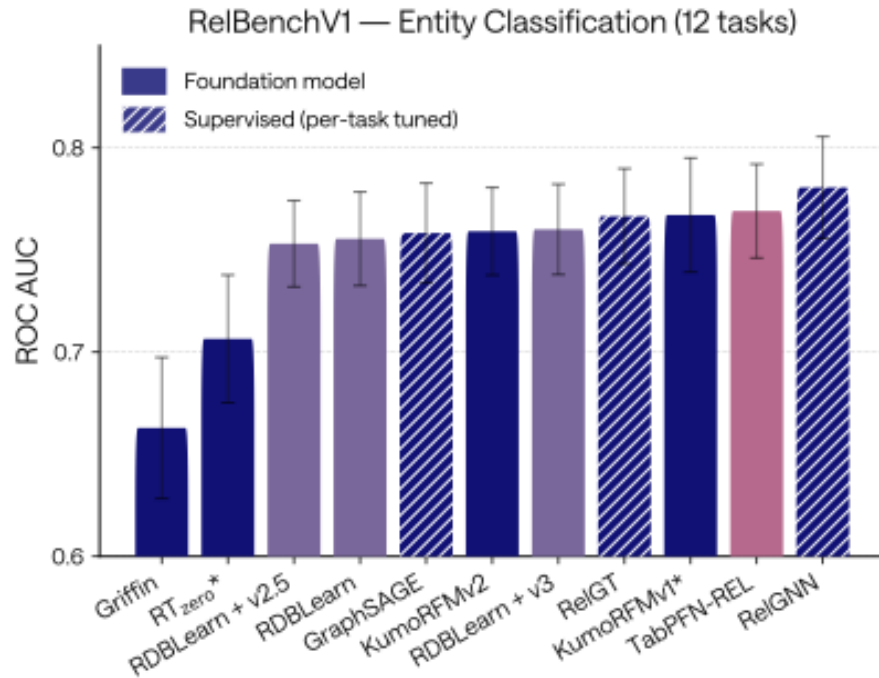
TabPFN v3

- **TabPFN v3** achieves better scalability than baselines



TabPFN v3

- **TabPFN-REL** achieves SOTA among relational foundation models
 - By applying TabPFN-3 to automatically flattened tables, but supervised RelGNN still performs better overall



Summary: TabPFN-v1, v2, and v3

- **Evolution of TabPFN**

- **v1**: Synthetic pretraining enables in-context prediction for small tabular tasks
- **v2**: Table-aware modeling improves performance on realistic tables
- **v3**: Efficient row-level ICL scales the paradigm to much larger datasets

- **Recent Research Directions**

- Make tabular ICL more scalable and efficient
- Improve pretraining with richer synthetic priors or real-world data
- Extend TabPFN to semantically rich and heterogeneous tables

Next: Recent TabPFN variants

TabDPT

- **Main Idea:** Using similar rows as context leads to better ICL prediction than using the full dataset and scalability
- **Method**
 - Treat each row as a single token (similar to TabPFN v1)
 - Retrieve the K most similar rows as context for ICL prediction
 - Pretrain on real-world tabular data with masked column SSL

TabDPT

- ICL retrieval and SSL leads to robust downstream generalization

Algorithm	CC18		CTR23	
	AUC	Accuracy	Correlation	R^2
TabDPT	0.933 $\pm$ [0.929, 0.937]	0.884 $\pm$ [0.882, 0.887]	0.837 $\pm$ [0.826, 0.848]	0.742 $\pm$ [0.731, 0.754]
TabPFN v2	0.932 $\pm$ [0.928, 0.936]	0.872 $\pm$ [0.869, 0.875]	0.835 $\pm$ [0.825, 0.845]	0.740 $\pm$ [0.729, 0.751]
TabPFN (kNN)	0.918 $\pm$ [0.915, 0.921]	0.850 $\pm$ [0.847, 0.853]	N/A	N/A
TabPFN	0.898 $\pm$ [0.895, 0.901]	0.812 $\pm$ [0.810, 0.814]	N/A	N/A
TabDPT (no val.)	<u>0.928</u> $\pm$ [0.924, 0.932]	0.874 $\pm$ [0.871, 0.877]	<u>0.835</u> $\pm$ [0.826, 0.844]	<u>0.739</u> $\pm$ [0.729, 0.749]
TabM	0.917 $\pm$ [0.913, 0.921]	<u>0.878</u> $\pm$ [0.876, 0.881]	0.824 $\pm$ [0.815, 0.834]	0.732 $\pm$ [0.722, 0.742]
TabR	0.925 $\pm$ [0.922, 0.928]	0.874 $\pm$ [0.871, 0.877]	0.828 $\pm$ [0.814, 0.842]	0.714 $\pm$ [0.693, 0.735]
MLP-PLR	0.912 $\pm$ [0.905, 0.919]	0.869 $\pm$ [0.865, 0.872]	0.829 $\pm$ [0.821, 0.838]	0.716 $\pm$ [0.699, 0.732]
MLP	0.872 $\pm$ [0.867, 0.876]	0.809 $\pm$ [0.805, 0.813]	N/A	N/A
XGBoost	0.926 $\pm$ [0.922, 0.929]	0.869 $\pm$ [0.866, 0.872]	0.827 $\pm$ [0.818, 0.837]	0.711 $\pm$ [0.698, 0.725]
LightGBM	0.924 $\pm$ [0.920, 0.927]	0.862 $\pm$ [0.859, 0.866]	0.825 $\pm$ [0.816, 0.834]	0.713 $\pm$ [0.697, 0.729]
CatBoost	0.926 $\pm$ [0.923, 0.930]	0.864 $\pm$ [0.860, 0.867]	0.822 $\pm$ [0.808, 0.837]	0.703 $\pm$ [0.683, 0.723]

TabDPT

- TabDPT outperforms semi-supervised baselines
 - Semi-supervised setting: only subsets of train samples can have labels
 - TabDPT (semi) uses the model's own predictions on unlabeled data as pseudo-labels to expand the context for better ICL

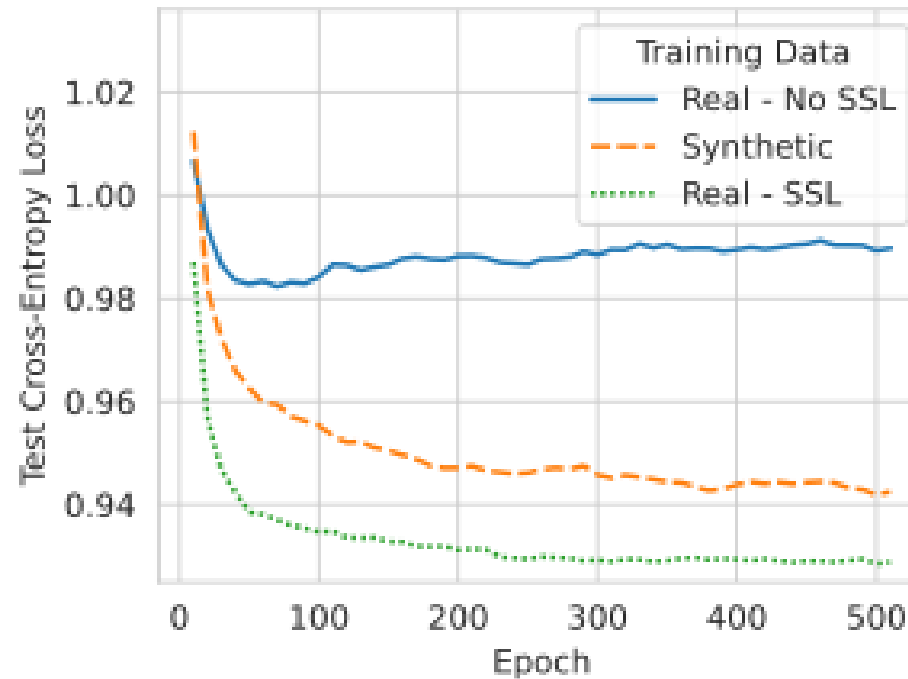
Method	cmc	karhunen	optdigit	diabetes	semeion	pixel	dna	Avg.
TabDPT (semi)	44.24	92.08	94.31	72.01	84.89	93.58	61.85	77.56
STUNT [48]	42.01	86.95	89.91	72.82	74.74	89.90	80.96	76.76
CACTUs [35]	42.14	85.48	87.92	70.75	68.22	87.21	84.40	75.16
VIME + LR[67]	37.92	86.63	89.63	66.56	77.66	88.71	74.73	74.55
kNN (STUNT) [48]	41.07	85.63	87.44	71.32	74.64	87.52	71.15	74.11
ICT [62]	38.00	88.25	90.84	67.63	74.67	89.13	69.55	74.01

Table 2: Few-shot accuracy on seven CC18 datasets. Only 10 labeled examples are available in each class of the training set, the rest are unlabeled.

TabDPT

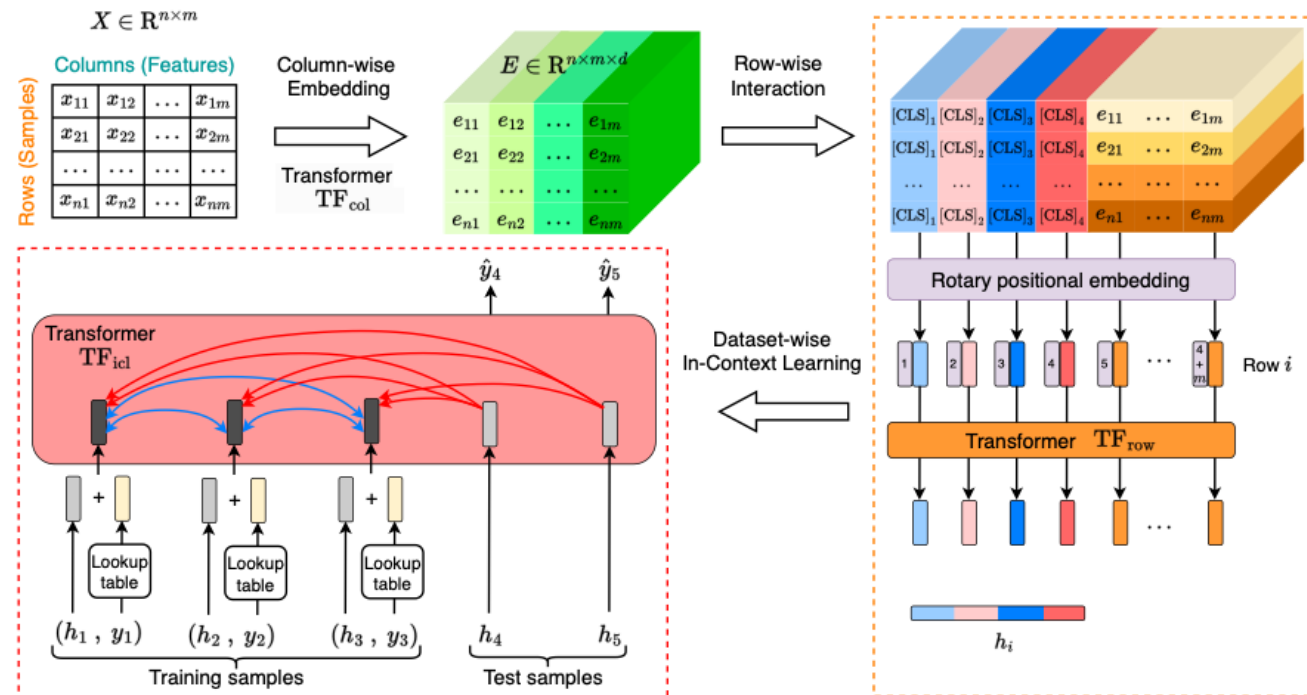
- **Key Findings**

- Pretraining on real-world data leads to faster convergence and better downstream accuracy than using purely open-source synthetic data generators



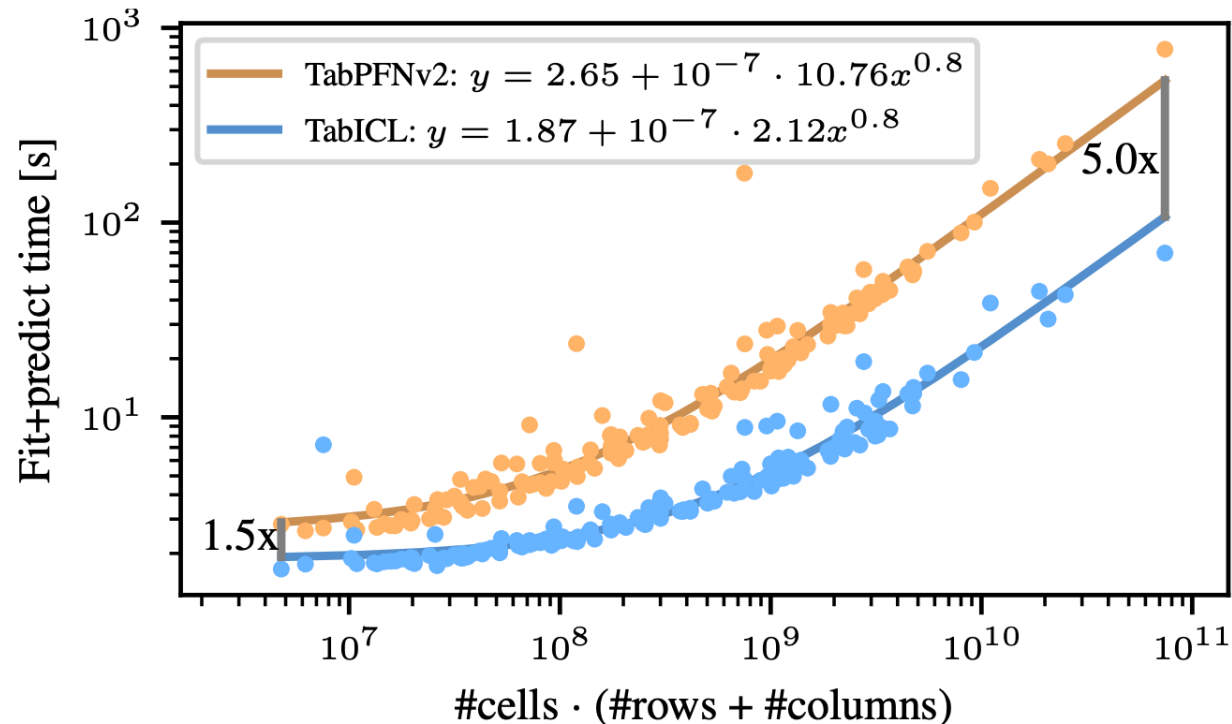
TabICL

- **Motivation:** TabPFN-v2's alternating row/column attention becomes costly on large tables
- **Method:** Compress feature information into fixed-dimensional row embeddings, then perform ICL over rows



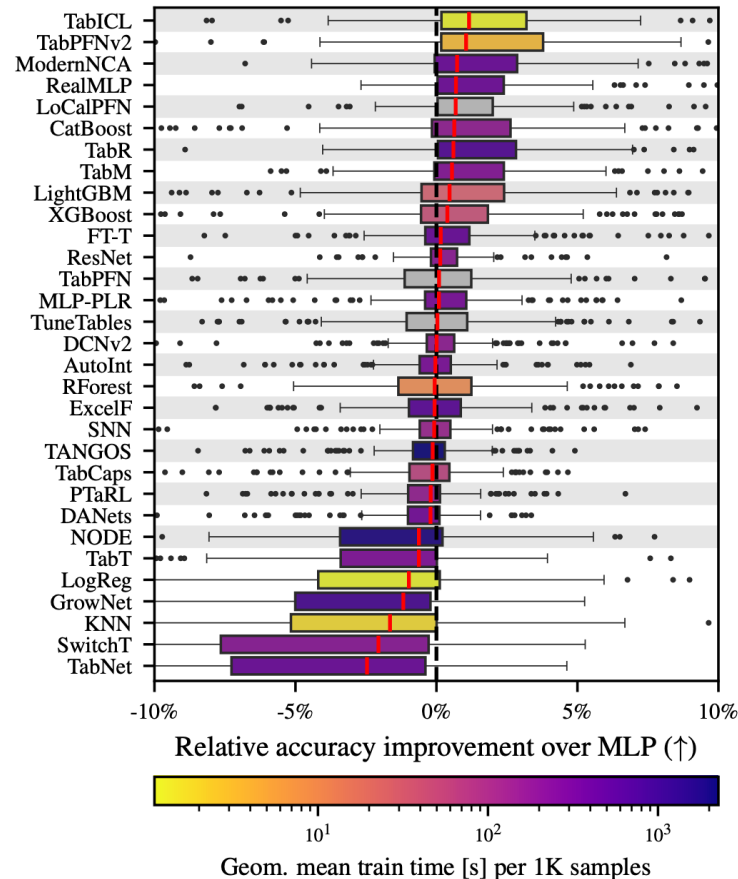
TabICL

- **TabICL** reduces computational cost via a two-stage framework
 - Given a tabular data with n rows and m column:
 - TabICL: separate two stage attention $O(m^2n + n^2)$
 - TabPFNV2: alternating 2D attention with $O(m^2n + mn^2)$



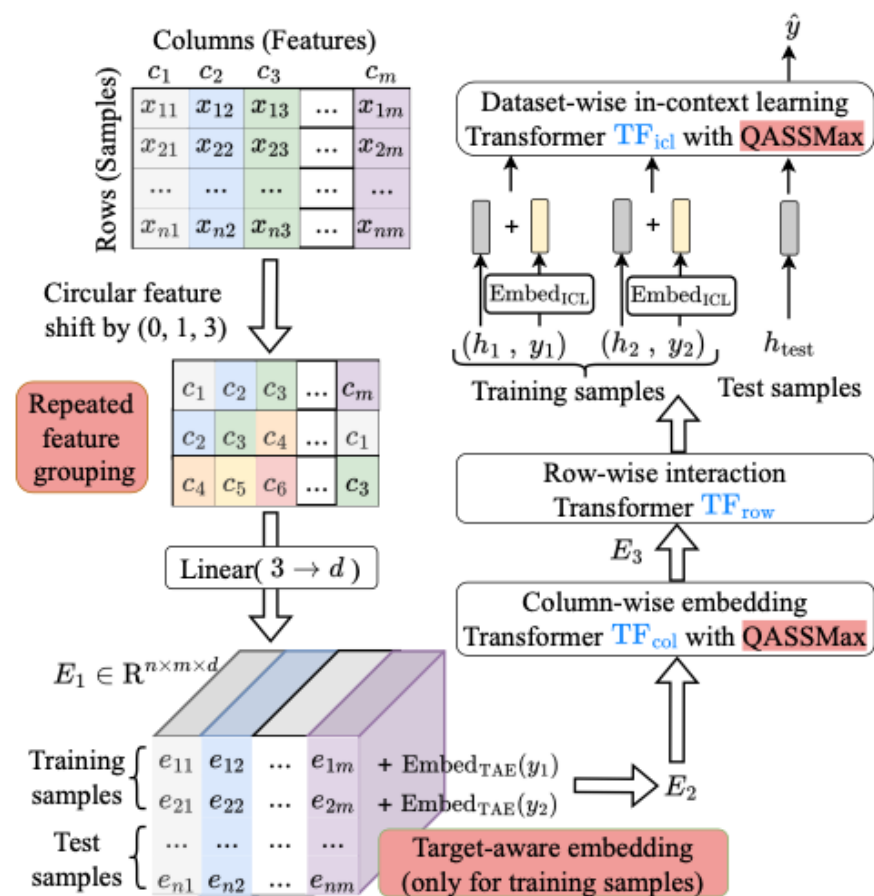
TabICL

- **TabICL** remains **competitive with TabPFN-v2** while **scaling better** to larger datasets



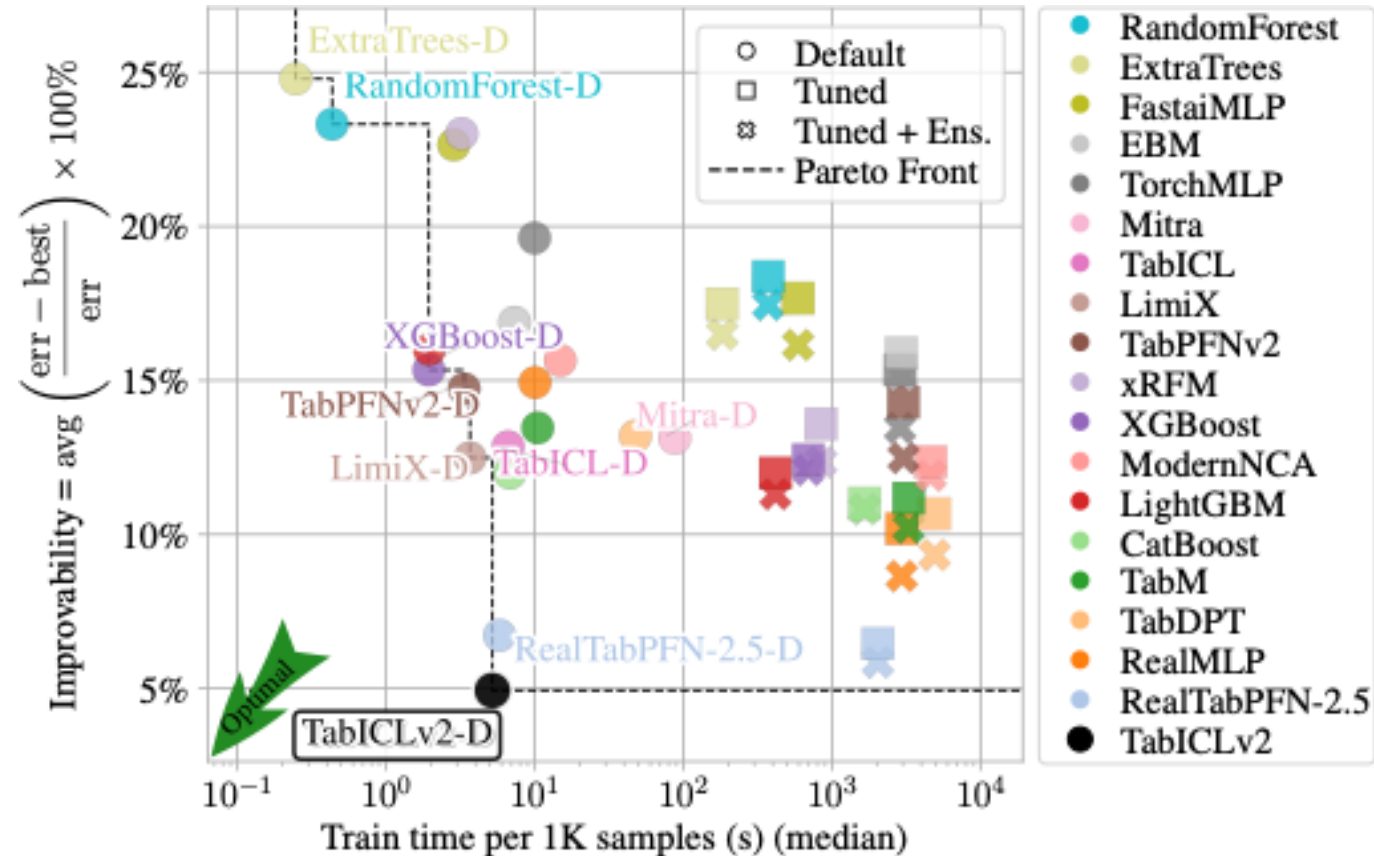
TabICLv2

- **Key changes:**
 - Richer **synthetic data generation** engine
 - **Circular-shift feature** grouping to reduce representation collapse
 - **Target-aware embedding** to inject label information early
 - **QASSMax** for scalable attention on long contexts
 - **Optimized pretraining protocol**, including Muon optimizer



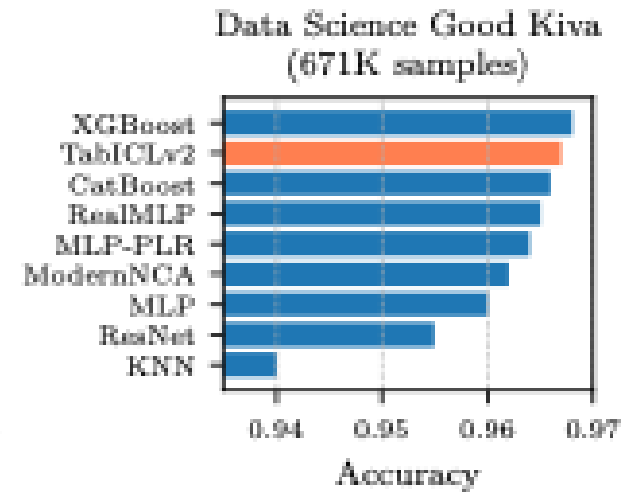
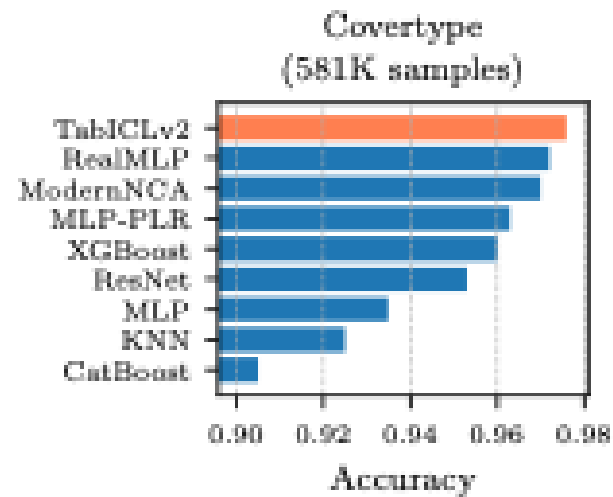
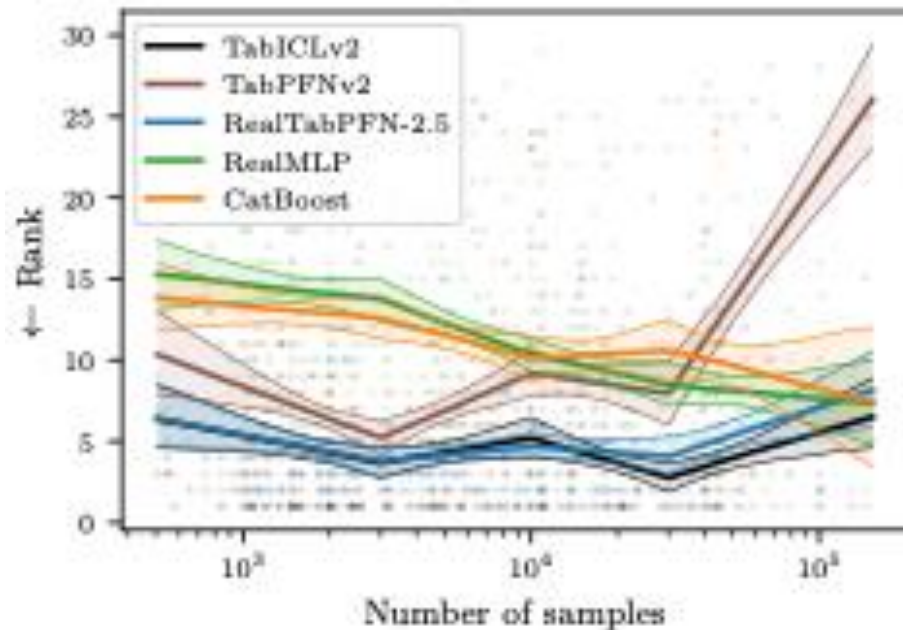
TabICLv2

- **Overall performance:** TabICLv2 outperforms recent baselines



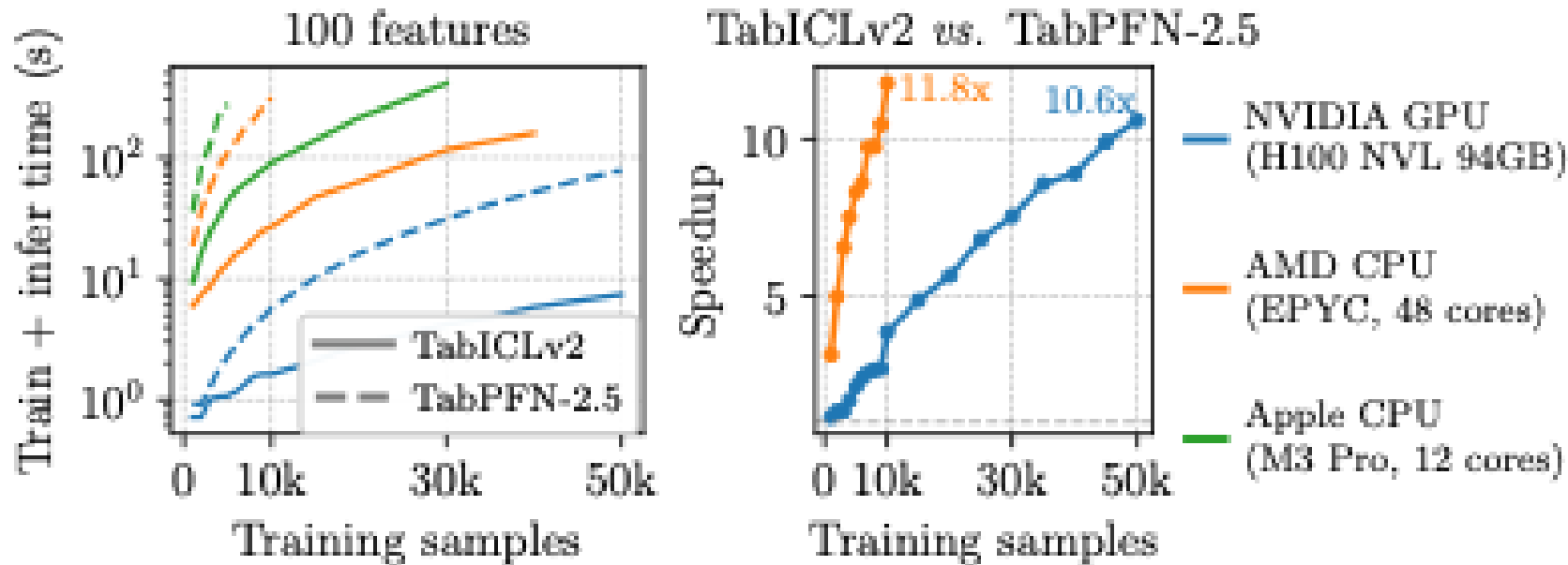
TabICLv2

- **Scalability:** TabICLv2 achieves scalability and the best performance on largest datasets



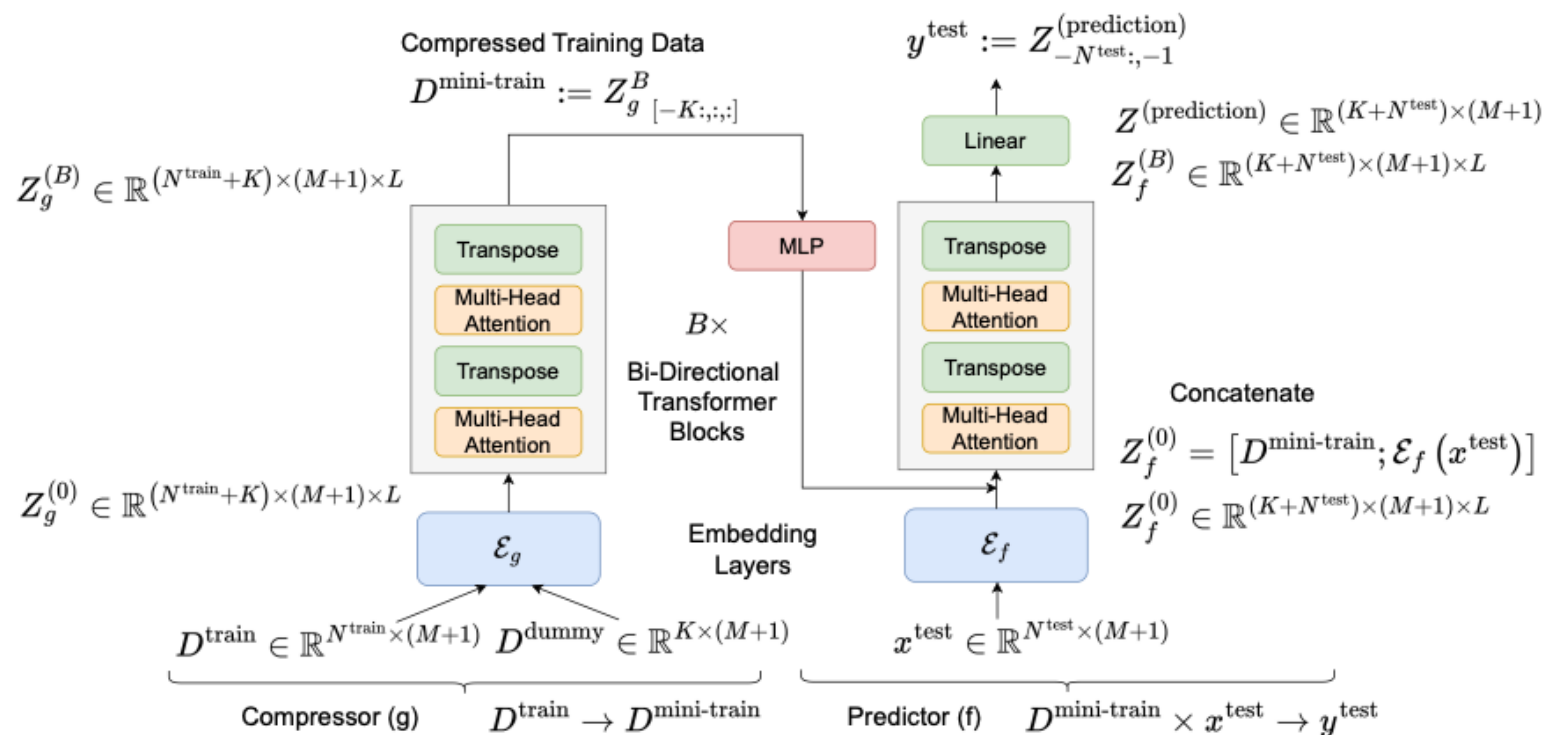
TabICLv2

- **Inference Speed:** TabICLv2 is consistently faster than TabPFN-2.5 on varying hardwares



TACO

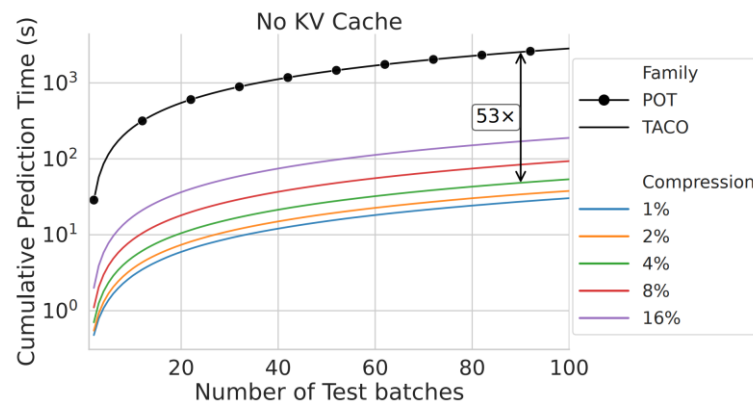
- **Key Idea:** Compress the context set before ICL
 - Learn a compact latent representation of training samples
 - Reduce the quadratic cost of attention over large context sets



TACO

- Achieves **53× speedup** and **94% memory reduction** over a predictor-only transformer, without significant performance drop

Model	Mean ROC AUC ($\uparrow$)
TabICL	0.866 ± 0.103
TabPFN V2.0	0.866 ± 0.103
POT	0.859 ± 0.102
TACO (r=1%)	0.854 ± 0.099
TACO (r=2%)	0.854 ± 0.099
TACO (r=4%)	0.855 ± 0.099
TACO (r=8%)	0.855 ± 0.099
TACO (r=16%)	0.855 ± 0.099

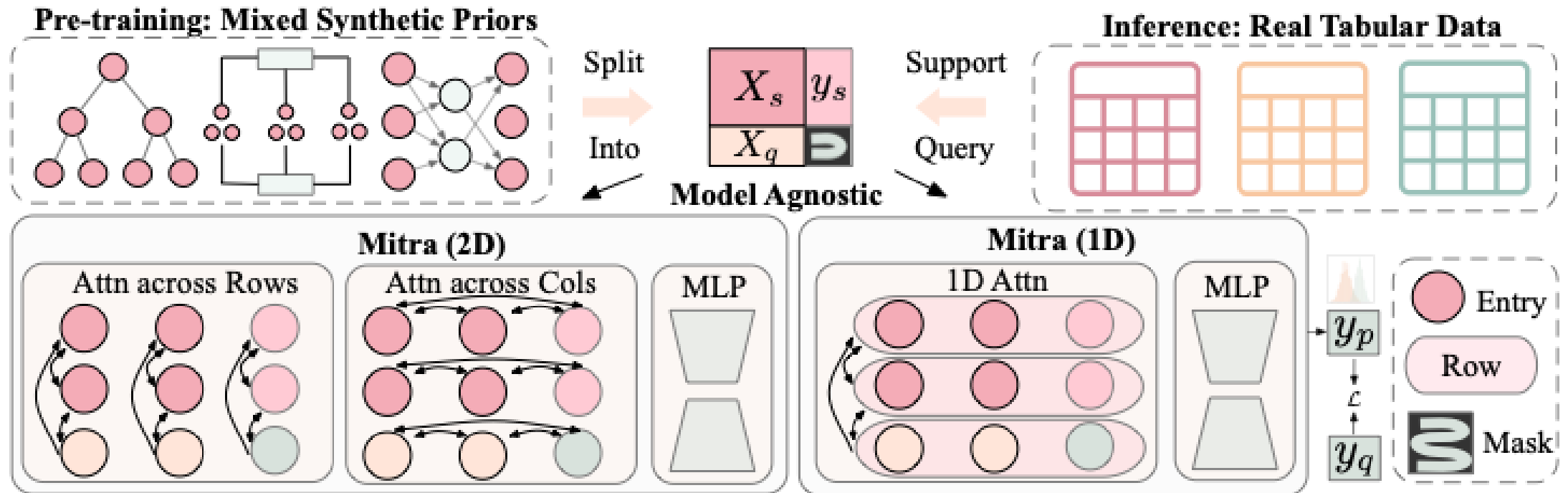


Method	Fit Time	First Predict Time	Subsequent Predict Time	Fit Mem	Predict Mem	
			Mean $\pm$ Std		Mean	Median
POT	8.41s	28.55s	28.67s $\pm$ 0.05s	36MB	22.45GB	22.45GB
TACO 1%	8.22s	29.68s	306ms $\pm$ 18ms ($\times$ 93.6)	92MB	776MB (–96.6%)	549MB (–97.6%)
TACO 2%	8.30s	29.80s	382ms $\pm$ 18ms ($\times$ 75.2)	92MB	1.05GB (–95.3%)	845MB (–96.3%)
TACO 4%	9.86s	30.71s	544ms $\pm$ 17ms ($\times$ 52.7)	92MB	1.63GB (–92.8%)	1.41GB (–93.7%)
TACO 8%	10.02s	32.47s	943ms $\pm$ 17ms ($\times$ 30.4)	92MB	2.78GB (–87.6%)	2.56GB (–88.6%)
TACO 16%	9.20s	35.36s	1.91s $\pm$ 0.01s ($\times$ 15)	92MB	5.10GB (–77.3%)	4.89GB (–78.2%)

Parentheses: $\times$ speed-up, –% memory reduction (vs. POT).

MITRA

- **Key Idea:** Improve TabPFN-style models by diversifying the synthetic pretraining prior
 - Instead of relying on a single prior, mix multiple synthetic task generators
 - Show that **prior design matters as much as model architecture**



MITRA

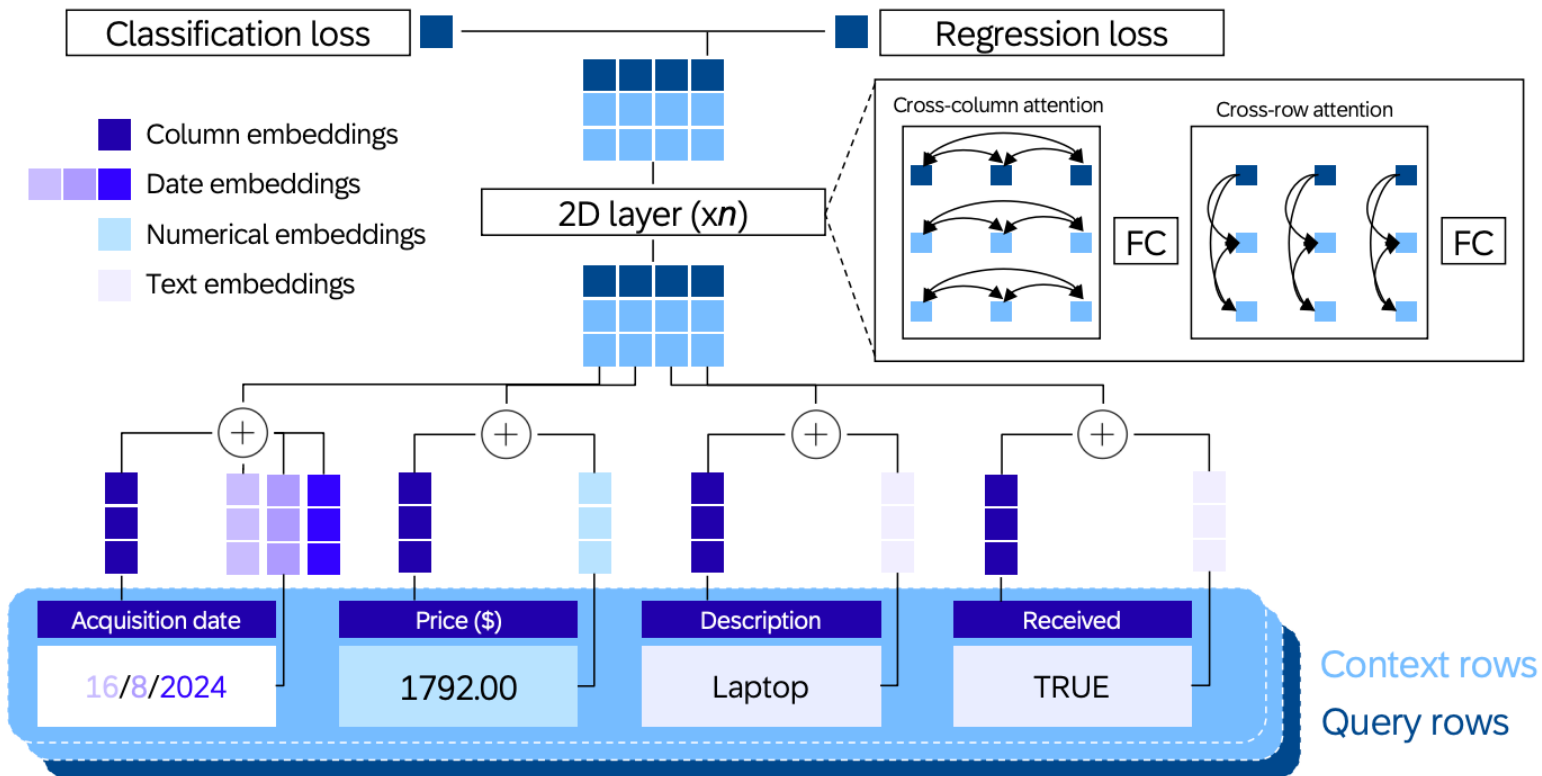
• Key Findings

- SCM prior is a strong standalone prior
- Mixing diverse priors improves robustness

Model	Ranking Metrics					Aggregated Metrics		
	Avg. Rank ↓	Elo ↑	Winrate ↑	RAcc ↑	CΔ ↓	AUC ↑	ACC ↑	CE ↓
RF	15.9	815 _(+7/-6)	0.25	0.38	42.8	0.831 _(0.147)	0.782 _(0.177)	0.521 _(0.468)
DT	15.4	840 _(+6/-7)	0.28	0.43	42.0	0.839 _(0.148)	0.791 _(0.177)	0.523 _(0.532)
GB	15.0	855 _(+6/-6)	0.3	0.52	38.3	0.843 _(0.147)	0.796 _(0.177)	0.501 _(0.464)
ET	14.3	883 _(+6/-6)	0.36	0.58	36.2	0.847 _(0.140)	0.803 _(0.169)	0.503 _(0.579)
DSRF	13.9	899 _(+6/-6)	0.36	0.58	34.5	0.849 _(0.144)	0.799 _(0.175)	0.473 _(0.410)
SCM	11.1	1000 _(+5/-6)	0.5	0.68	25.5	0.857 _(0.142)	0.812 _(0.176)	0.416 _(0.378)
SCM + RF	10.9	1005 _(+5/-6)	0.5	0.71	25.2	0.858 _(0.143)	0.816 _(0.173)	0.413 _(0.374)
SCM + DT	10.3	1025 _(+5/-5)	0.53	0.73	24.5	0.861 _(0.135)	0.816 _(0.169)	0.412 _(0.369)
SCM + GB	10.2	1028 _(+6/-5)	0.54	0.73	22.3	0.859 _(0.142)	0.817 _(0.172)	0.411 _(0.374)
SCM + DSRF	9.4	1058 _(+6/-5)	0.58	0.75	21.3	0.862 _(0.140)	0.819 _(0.171)	0.407 _(0.371)
SCM + ET	9.3	1063 _(+5/-5)	0.59	0.75	20.7	0.866 _(0.133)	0.823 _(0.169)	0.398 _(0.371)
SCM + ET + DT	10.2	1028 _(+5/-5)	0.54	0.74	22.4	0.869 _(0.129)	0.824 _(0.168)	0.403 _(0.371)
SCM + ET + RF	9.9	1042 _(+6/-5)	0.56	0.75	22.4	0.870 _(0.129)	0.825 _(0.166)	0.401 _(0.369)
SCM + ET + DSRF	9.6	1050 _(+5/-5)	0.57	0.74	22.1	0.860 _(0.139)	0.815 _(0.172)	0.410 _(0.373)
SCM + ET + GB	8.9	1076 _(+5/-6)	0.6	0.77	20.9	0.870 _(0.129)	0.827 _(0.164)	0.396 _(0.368)
SCM + ET + GB + DSRF	9.7	1046 _(+6/-6)	0.56	0.73	22.2	0.860 _(0.142)	0.817 _(0.171)	0.408 _(0.366)
SCM + ET + GB + RF	9.6	1050 _(+6/-6)	0.57	0.75	22.2	0.868 _(0.131)	0.823 _(0.167)	0.403 _(0.367)
SCM + ET + GB + DT	9.4	1059 _(+6/-5)	0.58	0.76	21.1	0.870 _(0.130)	0.826 _(0.165)	0.397 _(0.369)
SCM + ET + GB + DT + DSRF	9.5	1053 _(+5/-5)	0.57	0.75	22.9	0.859 _(0.140)	0.813 _(0.172)	0.411 _(0.368)
SCM + ET + GB + DT + RF	9.3	1062 _(+6/-5)	0.59	0.76	21.5	0.866 _(0.134)	0.823 _(0.167)	0.402 _(0.369)
SCM + ET + GB + DT + RF + DSRF (MITRA)	9.3	1062 _(+6/-6)	0.59	0.77	21.4	0.868 _(0.133)	0.822 _(0.168)	0.403 _(0.369)

ConTextTab

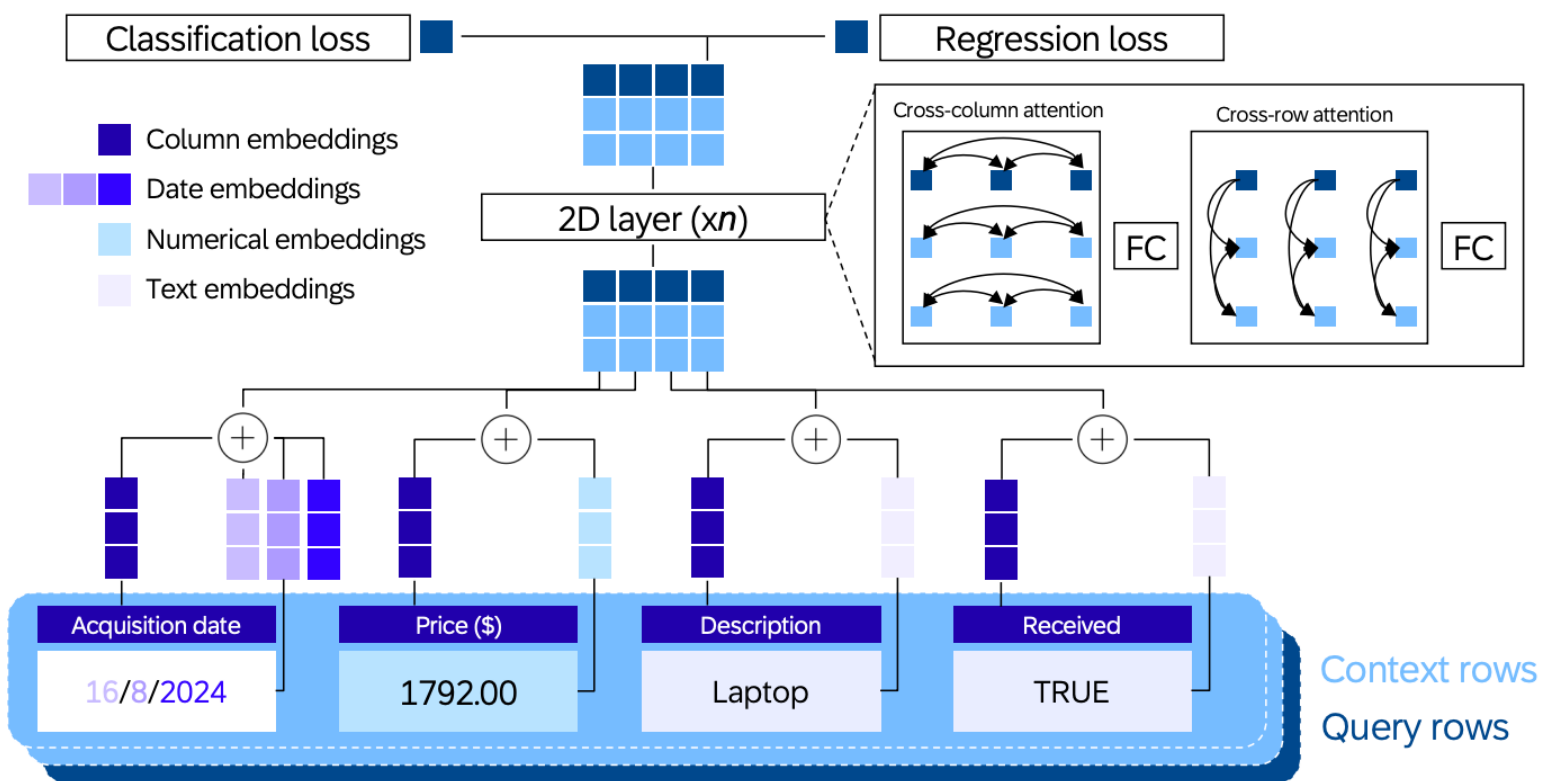
- **Main Idea:** Incorporate semantic information into in-context learning
 - Use pretrained embeddings for column names & textual/categorical values
 - Target semantically rich real-world tables



ConTextTab

- **Handling heterogeneous feature types**

- Text/categorical: pretrained text embedding model (all-MiniLM-L6-v2)
- Date: day/month/year separately embedded and summed



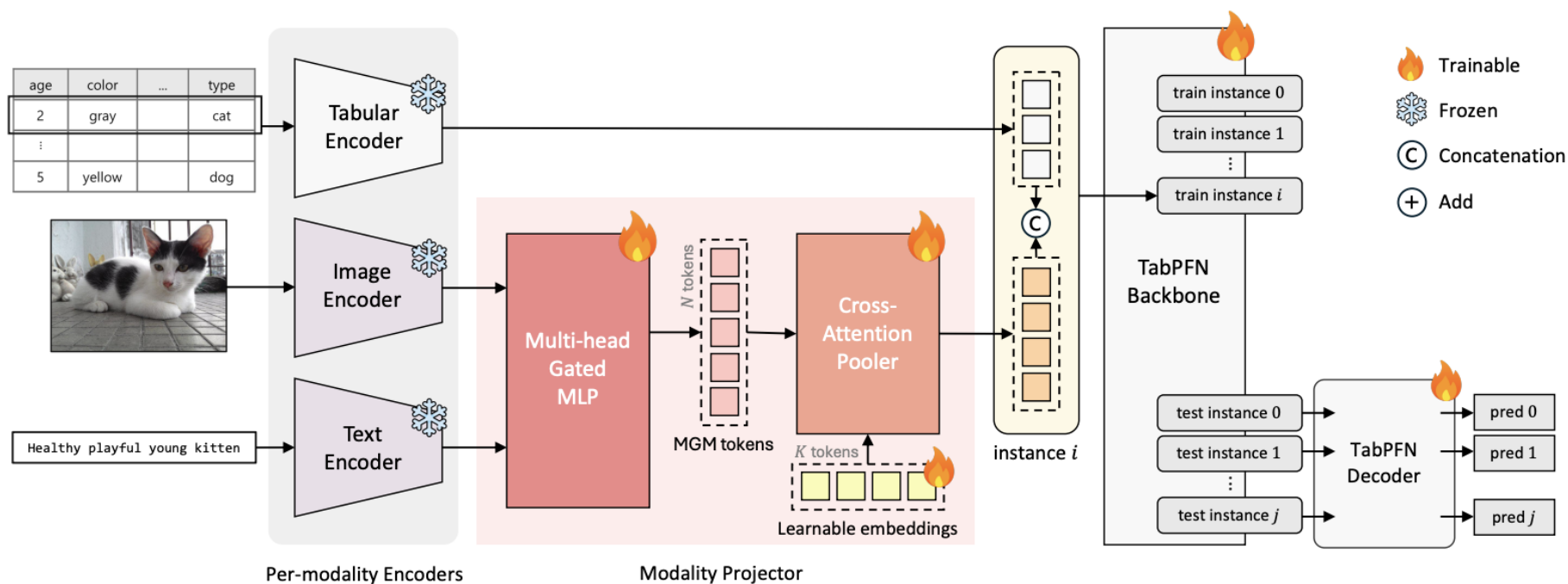
ConTextTab

- ConTextTab achieves SOTA on the **semantically rich** CARTE benchmark

Model Name	All	CARTE			OML-CC18		OML-CTR23		TabReD			TALENT-Tiny		
	Rank	Rank	Acc	R ²	Rank	Acc	Rank	R ²	Rank	Acc	R ²	Rank	Acc	R ²
AutoGluon [v1.2]	N/A	N/A	78.7	73.8	N/A	88.5	N/A	68.6	N/A	86.0	64.6	N/A	87.9	73.7
ConTextTab	2.96	1.55	76.9	72.4	3.56	86.6	3.89	73.3	4.12	85.4	63.2	2.59	87.6	76.1
LightGBM [HPO, CV]	2.85	3.00	73.4	67.5	2.94	87.5	2.91	68.8	1.25	86.0	64.9	2.73	86.9	74.1
CatBoost [HPO, CV]	2.71	3.06	75.9	68.2	2.72	87.7	2.74	68.8	2.00	86.0	64.3	2.32	87.2	76.4
RealMLP [HPO, CV]	2.50	3.20	73.4	67.5	2.35	88.3	1.89	75.0	1.88	86.0	64.6	2.54	88.3	76.4
TabM [HPO, CV]	2.98	3.71	73.2	66.9	2.93	87.9	2.37	74.3	3.12	86.1	64.2	2.62	87.5	75.7
XGBoost [HPO, CV]	3.11	3.78	73.1	66.7	2.83	87.5	3.23	70.9	1.25	86.0	64.9	3.00	86.8	73.6
TabPFN [v2.1.0]	3.63	5.76	72.3	65.0	2.89	87.4	3.29	75.1	4.25	85.6	63.8	2.32	87.8	74.4
TabDPT [v1.1, k=2048]	4.18	5.98	72.7	65.1	3.25	87.8	3.17	73.2	6.88	83.0	60.9	3.89	86.7	74.8
HistGradBoost	5.27	6.27	72.5	64.8	4.40	86.1	7.06	65.5	2.38	85.9	63.9	4.51	86.3	67.6
Random forest	6.22	8.00	71.5	63.3	5.33	85.7	6.80	67.9	6.00	85.4	60.7	5.00	85.8	70.6
Naive	10.51	11.0	53.0	-1.8	10.28	47.0	10.66	-8.4	9.75	80.8	-0.6	10.32	53.4	-19.2
CARTE [v0.0.26]	N/A	N/A	76.1	68.5	N/A	N/A	N/A	N/A	N/A	N/A	N/A	N/A	N/A	N/A
TabICL [v0.1.1]	N/A	N/A	72.5	N/A	N/A	88.0	N/A	N/A	N/A	85.1	N/A	N/A	87.4	N/A

MultiModalPFN

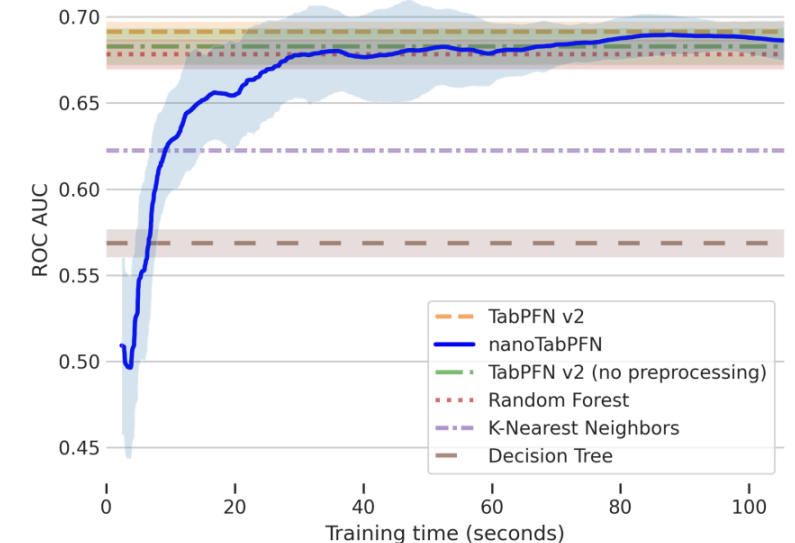
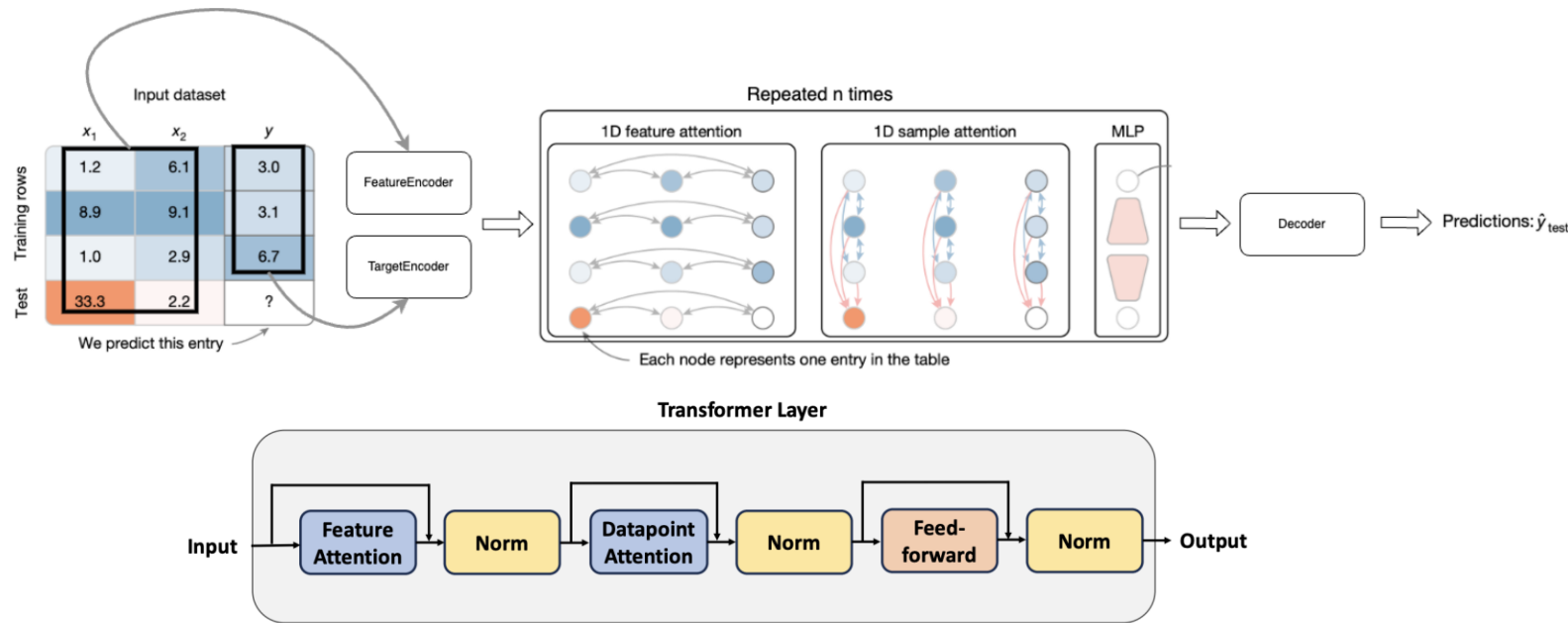
- **Key Idea:** Extend TabPFN to handle tabular data with images or text
 - **Encode non-tabular inputs** and project them into TabPFN-compatible representations
 - Jointly use tabular and multimodal information for prediction



(a) Overall architecture

nanoTabPFN

- **Key Idea:** Provide a lightweight reimplementaion of TabPFN v2
 - Implement the core TabPFN v2 architecture and training loop in fewer than 500 lines of code
 - Enable pretraining within minutes on a single GPU for learning and rapid experimentation



Open Directions for Tabular FMs

- **Scalability and Efficiency**

- Million-scale inference is becoming possible, but memory, latency, and long-context processing remain important bottlenecks

- **Pretraining Distribution**

- Richer synthetic priors and real-world tables provide complementary signals for better generalization

- **Semantic and Heterogeneous Tables**

- Column names, textual values, missingness, and mixed feature types remain underexplored sources of information

- **Beyond Standard Flat Tables**

- Relational, temporal, multimodal, and causal settings remain promising directions for future research

Thank You for Your Attention!
